

 [learning-zone](#) / [docker-and-kubernetes-basics](#) Public[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#)[docker-and-kubernetes-basics](#) / [docker-practice.md](#) **learning-zone** Update docker-practice.md 43f09e2 · 5 years ago

406 lines (312 loc) · 11.4 KB

Docker Workshop

1. [Introduction](#)
2. [Images and Containers](#)
3. [Create and Start Containers](#)
4. [Logs, Stop and Kill Containers](#)
5. [Inspect and Remove Containers](#)
6. [Run a Command in a Running Container with exec](#)
7. [Creating Custom Image](#)
8. [Creating an Image from a Dockerfile](#)
9. [COPY and ADD commands](#)
10. [Dockerizing a Nodejs project and deploy in Docker-Hub](#)
11. [React and SQL-Server connectivity](#)
12. [Docker Compose](#)

1. Introduction

- [Containers and Virtual Machines](#)
- [Docker Hub](#)
- Docker Installation
- [VS Code docker plugin](#)

```
docker version          // display docker versi
docker info
docker run hello-world  // check image in local
```

[↑ back to top](#)

2. Images and Containers

```
docker container ls      // Display list of
docker ps                // Display list of
docker container ls -a   // Display list of
docker image ls          // Display list of
docker images            // Display list of
```

[↑ back to top](#)

3. Create and Start Containers

```
docker container run hello-world    // more
docker container run busybox        // It i
docker container run busybox ls     // disp
docker container ls -a              // disp
docker container run hello-world
docker container create hello-world // retu
docker container ls -a
docker container start -a <container-id> // Sta
docker conatiner ls -a
docker system prune --all           // Rem
```

[↑ back to top](#)

4. Logs, Stop, and Kill Containers

```
docker container ls
docker container create busybox ls
docker container start <container-id>
docker container logs <container-id>
ping google.com
docker container create busybox ping google.com
docker container ls -a
docker container start <container-id>
```

```
docker container ls -a
docker container stop <container-id>    // Doe
docker container kill <container-id>    // Sto
```

[↑ back to top](#)

5. Inspect and Remove Containers

```
docker container ls -a
docker container inspect <container-id> // In
docker container ls -a
docker container rm <container-id>      // Re
docker container rm -f <container-id>   // Fo
docker system prune --all
```

[↑ back to top](#)

6. Run a Command in a Running Container with exec

```
docker container run redis
new terminal > docker container ls -a
docker container exec <container-id> redis-cli
docker container exec -it <container-id> redis-
# help
# exit
docker container exec -it <container-id> sh
# cd /
# ls
# exit
docker container run -it busybox sh    // Op
# ls -la                               // Sh
# exit
```

[↑ back to top](#)

7. Creating Custom Image

```
docker container run -it --name scm-container a
# apk add --update redis
# exit
```



```
docker container commit scm-container scm-image
docker image ls
docker container run scm-image redis-server
docker container exec -it <container-id> redis-
```

[↑ back to top](#)

8. Creating an Image from a Dockerfile

- Dockerfile

```
FROM alpine:latest
RUN apk add --update redis
CMD ["redis-server"]
```



```
docker build . // Build image
docker image ls // View build
docker image rm -f <image-id> // Remove build
docker image ls
docker build -t sofyspace/scm-redis:latest . /
docker run sofyspace/scm-redis
```



[↑ back to top](#)

9. COPY and ADD Commands

- Dockerfile

```
# From alpine library
FROM alpine
```



```
# Copy all the files from source directory to Docker image
COPY ./html_files /app/html
```

```
# Copy Text file to Docker image
COPY sample.txt /app/sample.txt
```

```
# Copy tar file to Docker image
ADD file.tar /app
```

```
# Copy svg file directly from url to Docker ima
ADD https://cdnjs.cloudflare.com/ajax/libs/line
```

```
docker build .           // Build image
docker image ls
docker run -it <image-id> sh // Run the build
# ls -la                 // Display all
# cd app/                 // app director
# ls -la                 // List all fil
# cd ..                   // Parent direc
```



[↑ back to top](#)

10. Dockerizing a Nodejs project and deploy in Docker-Hub

```
npm init // create package.json file
```

- index.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Node.js Server Running!');
});

app.listen(3000, () => {
  console.log(`Node APP Listening at http://lo
});
```



- package.json

[docker-and-kubernetes-basics](#) / docker-practice.md

↑ Top

Preview

Code

Blame

Raw



```
"scripts": {
  "start": "node index.js"
}
```

- **Dockerfile**

```
# alpine will download only basic version of node
FROM node:alpine
```

```
# Instead of root directory, program will use "
WORKDIR /usr/app
```

```
# Copy local directory to nodejs directory
COPY ./ ./
```

```
# Perform npm install
RUN npm install
```

```
# Run npm start in command prompt
CMD ["npm", "start"]
```



```
docker build -t sofyspace/scm-website:latest .
docker run sofyspace/scm-website
docker run -p 3000:3000 sofyspace/scm-website
docker login
docker push sofyspace/scm-website
```



[↑ back to top](#)

11. Run SQL Server on Docker

```
docker pull mcr.microsoft.com/mssql/server:2017
docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=<
```



Parameter	Description
-e "ACCEPT_EULA=Y"	End-User Licensing Agreement
-e "SA_PASSWORD= YourStrong@Passw0rd "	Required setting for the SQL Server image.

Parameter	Description
<code>-p 1433:1433</code>	Map a TCP port on the host environmer (first value) with a TCP port in the container (second value).
<code>--name sql1</code>	Specify a custom name for the container rather than a randomly generated one
<code>-h sql1</code>	Used to explicitly se the container hostname
<code>mcr.microsoft.com/mssql/server:2017-CU8-ubuntu</code>	The SQL Server 2017 Ubuntu Linux container image.

Push SQL-Server to Docker Hub:

Dockerfile:

```
FROM mcr.microsoft.com/mssql/server:2017-CU8-ub 
```

```
# Create work directory
```

```
RUN mkdir -p /usr/work
```

```
WORKDIR /usr/work
```

```
# Copy all scripts into working directory
COPY . /usr/work/

# Grant permissions for the import-data script
RUN chmod +x /usr/work/import-data.sh
EXPOSE 1433
CMD /bin/bash ./entrypoint.sh
```

```
// Build SQL Server Image
docker build -t sofyspace/sqlserver:dev .

// Push to Docker Hub
docker push sofyspace/sqlserver:dev

// Connect to Sql Server
docker exec -it sqlserver "bash"

// Start the Sql Server Container
docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=<
```



Connect to SQL Server:

```
docker exec -it sql1 "bash"
/opt/mssql-tools/bin/sqlcmd -S localhost -U SA
```



Create a new database:

```
CREATE DATABASE TestDB
SELECT Name from sys.Databases
GO
```



Insert Data:

```
USE TestDB
CREATE TABLE Inventory (id INT, name NVARCHAR(50))
INSERT INTO Inventory VALUES (1, 'Banana', 150)
GO
```



Select Data:

```
SELECT * FROM Inventory WHERE quantity > 152;  
GO
```



Reference:

- <https://docs.microsoft.com/en-us/sql/linux/quickstart-install-connect-docker?view=sql-server-ver15&pivots=cs1-powershell>

[↑ back to top](#)

12. Docker Compose

- Dockerfile

```
FROM node:10
```



```
WORKDIR /usr/src/app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["npm", "start"]
```

- docker-compose.yml

```
version: '3'
```



```
services:
```

```
  app:
```

```
    container_name: docker-node-mongo
```

```
    restart: always
```

```
    build: .
```

```
    ports:
```

```
      - '80:3000'
```

```
    external_links:
```

```
      - mongo
```

```
  mongo:
```

```
    container_name: mongo
```

```
    image: mongo
```

ports:

- '27017:27017'

```
docker-compose up    // Install nodejs and mong   
localhost            // Default port is 80  
docker-compose down  // Remove Container
```



Reference:

- <https://github.com/bradtraversy/docker-node-mongo>

[↑ back to top](#)